

第一阶段工作详解—看完你就能汇报

纪文龙 2026 年 4 月 10 日

GitLab 分支: `jiwenlong/vectorize-pipeline` 提交: `2a2b473`

本文档用途：用大白话帮你搞懂你「做了什么」和「组会怎么讲」。每个专业术语都有解释框，看不懂就看蓝框。

第一部分：你到底做了什么

1 一句话总结

你做了一个**自动化处理流程**，效果是：给它一篇论文的 OCR 标注数据，它能自动存起来，然后你问它问题，它能找到论文中最相关的段落回答你。

什么是 Pipeline（管线/流水线）？

Pipeline 就是「一步一步处理的流水线」，像工厂的生产线一样：原材料进去 → 经过几道工序 → 出成品。在我们这里：

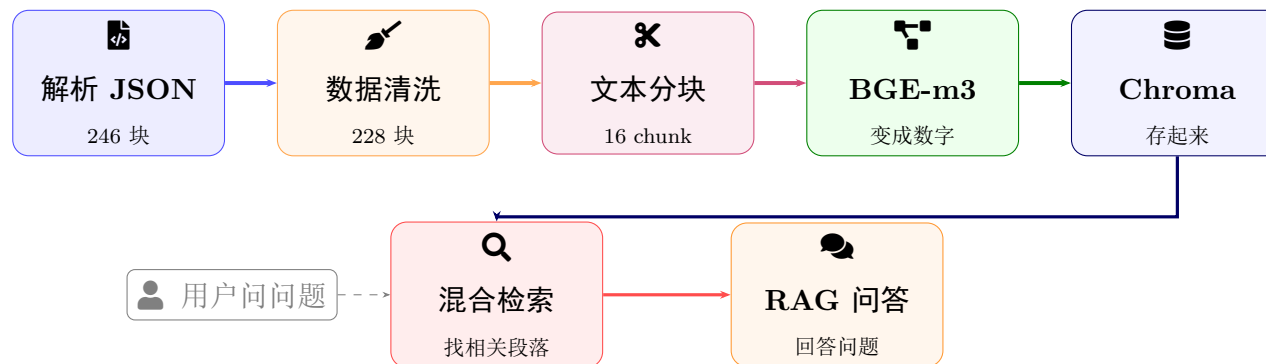
原材料 = 标注好的 JSON 文件（论文的 OCR 识别结果）

成品 = 一个能回答问题的系统

汇报时你可以说：「我做了一个数据处理的流水线」，不用说 Pipeline 这个词也完全可以。

2 整体流程图

下面这张图就是你的「流水线」，从左到右依次经过 6 个步骤：



每一步是什么意思？下面逐个解释。先说一个贯穿全文的词：

什么是「文本块」？

一篇论文的 PDF 扫描件上有很多段文字。郭继鸿用标注工具在每一段文字上画了一个框，每个框里的文字就是一个**文本块**。比如一个标题是一个块，一个段落是一个块，一个列表项也是一个块。4 页的论文一共画了 246 个框，所以就有 246 个文本块。你可以把「文本块」理解为「一小段文字」。

什么是 OCR？

OCR = 光学字符识别。就是把纸质文件或 PDF 扫描件上的文字，用软件自动识别成电脑能编辑的文字。我们项目里的论文都是扫描版 PDF（相当于照片），电脑看不懂里面的文字，需要 OCR 先把文字「读」出来，郭继鸿用 Label Studio 做的就是这个事情。

3 第 1 步：解析 JSON

什么是 Label Studio JSON?

郭继鸿用一个叫 Label Studio 的标注工具，在 PDF 论文的扫描图片上画框，标记出「这段是标题」「这段是正文」「这段是列表」。标记完成后导出一个 JSON 文件（一种数据格式，类似于 Excel 表格但更灵活）。这个 JSON 文件就是我们的**原材料**。

- 这一步做了什么：**读取 JSON 文件，把里面的文字提取出来。每一条标注包含：
- 位置信息（在页面上的坐标）
 - 类型标签（Title= 标题 / Para= 段落 / List= 列表项）
 - 文字内容（OCR 识别出的文字）
- 一个改进：**原版代码只处理了「标题」和「段落」两种标签，**漏掉了「列表」**（就是论文里的编号列表，比如 1. 2. 3. 这种），有 15 个列表段落被跳过了。我的代码补上了。
- 结果：**从 1 篇论文（4 页）中提取出 246 个文本块（= 246 小段文字）。

4 第 1.5 步：数据清洗

什么是数据清洗？

原始数据里有「脏数据」——不完整、有错误、质量差的数据。数据清洗就是把这些脏数据修好或删掉，就像洗菜一样，把烂叶子摘掉。

发现了什么问题？

因为这篇论文是**双栏排版**（左右两列文字），OCR 在画框时把一些句子切断了，产生了碎片：

碎片	页码	怎么回事
“程体系。”	第 1 页	「课程体系。」被切成两半，只留了“程体系。”
“建方法”	第 1 页	「构建方法」被切成两半
“印证。”	第 2 页	句子末尾的碎片

怎么修的？

我写了一个函数：如果一个文本块不足 10 个字，就自动合并到旁边的块里。

效果：246 块 → 228 块，合并了 18 个碎片。之后质量检测显示 0 个问题。

5 第 2 步：文本分块

什么是分块（Chunking）？

228 个小块太碎了，直接存不好检索。分块就是把很多小块拼成合适大小的「段落」——大约 500 个字符一段（约 250 个中文字），既不太长也不太短。拼好的段落叫 **chunk**。

为什么要这个大小？太短的话信息不够，大模型理解不了上下文；太长的话搜索时噪音太多，无关内容太多。500 字符是业界常用的经验值。

一个关键改进：标题触发新段落

如果不做特殊处理，可能出问题：

❌ 普通做法（有问题）

... 前面的正文正文正文...

一、新章节的标题

↑ 标题粘在上一段末尾了！

新章节的正文...

↑ 这段没有标题了！

VS

✅ 我的做法（正确）

... 前面的正文正文正文...

↑ 上一段干净地结束

一、新章节的标题

新章节的正文...

↑ 标题和正文在一起

为什么这很重要？如果有人搜「新章节的标题」，左边的做法会匹配到上一段（因为标题在那里），但上一段的内容跟这个标题没关系。右边的做法就不会搞混了。

结果：228 个小块拼成了 16 个 chunk，平均长度 429 字符。

6 第 3 步：向量化

什么是向量化（Embedding）？

电脑不认识中文，只认识数字。向量化就是把一段文字转成一串数字（叫做「向量」），这串数字能代表这段文字的**语义**（含义）。

意思相近的文字，转出来的数字也很接近。比如：

「燃气轮机状态监控」和「发动机健康管理」→ 数字很接近

「猫吃鱼」和「燃气轮机」→ 数字差很远

用什么模型做的？用的是 **BGE-m3**，北京智源研究院做的开源模型，目前中文检索效果最好的模型之一。每段文字会变成 1024 个数字（1024 维向量）。

什么是 Chroma？

Chroma 是一个**向量数据库**——专门用来存那些数字（向量）的数据库。存进去之后，你给它一个新的问题，它能快速找到数据库里跟这个问题最接近的几段文字。

类比：普通数据库像图书馆的编号检索（按书名、作者找书）；向量数据库像一个特别聪明的图书管理员（按「你想看什么内容的书」找书）。

这一步做了什么：

1. 加载 BGE-m3 模型
2. 把 16 个 chunk 都转成 1024 维的数字（向量化）
3. 存入 Chroma 数据库（保存在本地磁盘，下次不用重新算）

7 第 4 步：混合检索

这是最重要的升级。

什么是检索 (Retrieval)?

检索就是「搜索」。你输入一个问题，系统在数据库里找到跟这个问题最相关的几段文字返回给你。跟百度搜索类似，但搜的是我们自己存进去的论文内容。

基础版本只用了一种搜索方法（语义检索）。我的版本用了两种方法，然后把结果合并：



什么是 BM25? 什么是 jieba? 什么是 RRF?

BM25: 一个经典的搜索算法，跟百度早期的搜索原理类似。你搜“燃气轮机”，它就去数几篇文章里“燃气轮机”出现了几次，出现多的排前面。

jieba（结巴分词）：中文没有空格，电脑不知道从哪断词。jieba 就是一个切词工具，比如把”数据库系统”切成”数据库/系统”。BM25 需要先切词才能搜索。

RRF（Reciprocal Rank Fusion）：一种合并排名的方法。语义检索给出一个排名、BM25 给出一个排名，RRF 把两个排名合在一起，算出最终排名。简单理解：两种方法投票，得票多的排前面。

效果对比——为什么两种方法都需要：

搜索什么	语义检索得分	BM25 得分	说明
“燃气轮机健康管理”	0.685（高）	3.2	两种都能找到
“布鲁姆认知过程”	0.387（低）	4.7 （高）	语义找不到，BM25 能！
“船舶动力装置”	0.542 （高）	0.0（没有）	BM25 找不到，语义能！

汇报时你可以这样说：「两种方法各有优劣，合在一起可以覆盖各自的盲区。」

8 第 5 步：RAG 问答

什么是 RAG？

RAG = Retrieval-Augmented Generation，翻译成「检索增强生成」。

简单说：你问一个问题 → 先去数据库里**检索**相关段落 → 把段落喂给大模型（如 ChatGPT）→ 大模型根据这些段落**生成**一个答案。

为什么需要 RAG？ 直接问 ChatGPT，它不知道你论文里写了什么。但如果你先帮它找到相关段落，它就能基于论文内容回答了。

当前进度：检索部分做好了，大模型 API 还没接。接上之后就是完整的 RAG 系统。

当前能做的：输入一个问题，返回论文中最相关的 3 段原文 + 页码 + 相关度。后续接上智谱/千问的 API 就能自动生成答案。

使用方法（在命令行运行）

```
python 向量化存储.py --vectorize --ask "船舶燃气轮机课程知识体系如何构建？"
```

第二部分：组会怎么讲

9 开场（30 秒）

「这一周我主要完成了第一阶段的任务——从 JSON 数据的解析、清洗、分块、到向量化存储，全流程跑通了。另外我在基础功能之上做了三个优化：数据清洗（修复了 OCR 碎片问题）、混合检索（语义 + 关键词两路融合），以及搭好了 RAG 问答的框架。」

10 数据概况（1 分钟）

「当前测试数据是一篇关于船舶燃气轮机课程知识体系的论文，4 页，OCR 标注后有 246 个文本块。

我在跑的过程中发现两个数据质量问题。

第一个是 OCR 标注把一些句子切断了，比如“构建方法”被切成了两半，只剩下“建方法”3 个字，这种碎片会影响后面的检索质量。我在代码里加了自动合并——不足 10 个字的碎片自动拼到相邻块里，246 块清洗到了 228 块。

第二个问题是双栏 PDF 的左右栏文字交叉，这个需要拿到标注框的 x 坐标才能解决，我想问一下郭老师 JSON 里是否有这个字段。」

11 技术亮点（2 分钟）

「除了跑通基础流程，我额外做了三件事：

第一，数据清洗。把 OCR 碎片自动合并了，合并了 18 个碎片，质量检测从 7 个问题变成 0 个。清洗后分块更紧凑，chunk 从 21 个减少到 16 个。

第二，混合检索。基础版只有语义检索（就是把文字转成数字来找相似段落），我加了 BM25 关键词检索（直接按关键词匹配），然后用 RRF 把两路结果融合排序。举个例子：搜“布鲁姆认知过程”，语义检索找不太到，但关键词检索能直接命中。反过来搜“船舶动力装置”，关键词完全匹配不到，但语义检索能找到相关内容。

第三，端到端问答框架。现在可以输入一个问题，系统自动检索出最相关的段落和出处。后面接上大模型 API 就能自动写答案了，框架已经搭好。」

12 下一步计划（30 秒）

「下一步有三件事：

1. 等全量的 1000 本手册 JSON，把数据规模扩大

2. 接入大模型 API（智谱清言或者通义千问），实现真正的自动问答
3. 开始调研 Graph RAG，为后面的论文做准备

」

什么是 Graph RAG?

普通 RAG 只是把文字段落存起来搜索。Graph RAG 在此基础上加入了知识图谱——把实体和关系用图的方式连起来（比如「BGE-m3 → 是一种 → 向量化模型」），这样能做更精准的推理。这是我们项目后续要做的、也是拿来发论文的方向。

13 刘超可能问的问题

13.1 Q: 为什么用 BGE-m3 不用其他模型?

「BGE-m3 是目前中文语义检索效果最好的开源模型之一，在 MTEB 中文排行榜上排名靠前。它支持多语言，参数量不大，普通电脑就能跑，不需要 GPU 服务器。」

MTEB 是什么?

MTEB = Massive Text Embedding Benchmark，是评测向量化模型好不好的排行榜。类似于手机芯片的跑分排行。BGE-m3 在中文任务上排名靠前。

13.2 Q: 混合检索比纯语义检索好多少?

「在精确关键词查询上明显更好。比如搜“布鲁姆认知过程”，纯语义只有 0.387 的相似度，排不到前面。但 BM25 通过关键词精确匹配到了，得分 4.7 分，直接排第一。两种方法互补，覆盖了各自的盲区。」

13.3 Q: 为什么分块用 500 字符?

「这是经验值。太短信息不够，大模型理解不了上下文；太长噪音太多，搜出来一大段里面可能就一句话相关。500 字符大约 250 个中文字，差不多一个自然段。后面数据量大了可以做实验调优。」

13.4 Q: RRF 的 70/30 比例怎么定的?

「0.7 是业界常用的默认值。因为大多数情况下语义理解更重要，关键词只是补充。后续可以在测试集上调参。」

13.5 Q: 1000 本手册能跑得动吗?

「向量化是一次性的，1000 本手册估计几万个 chunk，可能需要几个小时跑完。但存好之后检索是毫秒级的。如果嫌慢可以用 GPU 加速。」

13.6 Q: 双栏文本交叉的问题解决了吗?

「碎片合并的问题解决了，但双栏文字交叉的问题还没有完全解决。需要拿到标注框的 x 坐标，用 x 坐标判断这段文字属于左栏还是右栏，然后分别排序。我想确认一下 JSON 数据里是否有 x 坐标。」

14 代码在哪

```
# GitLab 仓库（浏览器打开这个网址）
http://117.78.35.35/rag1/plant-graph-rag

# 你的分支（左上角下拉框切换到这个分支就能看到你的代码）
jiwenlong/vectorize-pipeline

# 你的代码文件
src/jiwenlong/pipeline.py

# 本地运行命令
cd 代码/
```

```
uv run python 源代码/纪文龙/向量化存储.py --vectorize --search
uv run python 源代码/纪文龙/向量化存储.py --vectorize --ask "你的问题"
```